

**School of Computing and Information Technology
University of Wollongong**

CSIT213 Java Programming

Assignment 3 (15%)

Due: refer to Moodle

Objectives

- To apply Object Oriented Design.
- To apply Object Oriented Programming.
- To implement solution in Java and IDE.

Overview

Modern cities deploy air quality sensors to track pollution levels such as PM2.5 concentration. In this assignment, you will build a Java program that:

- Loads sensor readings from a CSV file
- Validates and stores only correct readings
- Logs any invalid records to an error report
- Processes stored readings using a pluggable analyser
- Outputs analysis results to a results report

Scenario

A city has multiple air quality sensors across different districts (e.g., North, South). Each sensor records a daily PM2.5 concentration reading (in $\mu\text{g}/\text{m}^3$). The city environment officer wants:

- An error log of invalid lines in the CSV file.
- A daily average PM2.5 concentration per district.

You will implement an analyser that calculates:

- Average PM2.5 concentration grouped by date + district

Example key format: 2026-02-05_North

Classes and Interface

Your solution must include the following classes and interface (names must match):

AirQualityReading

Represents one valid sensor reading.

Attributes

- sensorId (String)
- district (String)
- readingDate (LocalDate)
- pm25 (double)

Rules

- readingDate must not be null
- pm25 must be between 0.0 and 500.0 inclusive
- If not, throw AirQualityDataException

AirQualityDataException

A custom checked exception used to reject invalid sensor data at object creation time.

Analyser (Interface)

Represents a processing strategy.

This allows different analysers to be “plugged in” without changing the main program.

DistrictAirQualityAnalyser

An analyser that computes:

- Average PM2.5 concentration per (date + district)
- Rounded to 2 decimal places

Output keys formatted: yyyy-MM-dd_DISTRICT

CityAirStat

Main application controller:

- Loads the data file
- Stores valid records
- Stores errors
- Runs an analyser
- Writes output files

The program reads from a csv file (e.g., data.csv).

Input file Format

Each data line contains:

- date,sensorId,district,pm25

Example:

- 2026-02-05,S001,North,35.2
- 2026-02-05,S002,North,40.6
- 2026-02-05,S010,South,55.0

We may assume there are no duplicate data.

Comments

Any line beginning with # is treated as a comment and must be ignored.

Example:

This is a comment line

2026-02-05,S001,North,35.2

Refer to Appendices for sample data.csv

Data Validation Requirements

When loading each line, validate the following:

Date

- Must not be empty
- Must match format: yyyy-MM-dd
- If invalid, record: Line X: Invalid Reading Date

Sensor ID

- Must not be empty
- If invalid, record: Line X: Invalid Sensor ID

District

- Must not be empty
- If invalid, record: Line X: Invalid District

PM2.5

- Must be a number (double)
- If not numeric: Line X: Invalid PM2.5
- If numeric but outside range (0.0 to 500.0):
 - Should be rejected by AirQualityReading constructor throwing AirQualityDataException
 - If invalid, record: Line X: Invalid PM2.5: <value>

Rule on Storing Data

- If any error occurs in a line, that line must not be added to reading data.

Output Files

errors.txt

After loading is finished, write every error message into errors.txt, one per line.

Example:

Line 3: Invalid Reading Date

Line 7: Invalid PM2.5

Line 12: Invalid District

results.txt

After processing the analyser, write each computed entry:

Format:

<date>_<district> : <averagePm25>

Example:

2026-02-05_North : 37.97

2026-02-05_South : 51.90

Refer to Appendices for sample errors.txt and results.txt

Test Code and automated grading:

- The testing code StudentAssertionTest.java is provided to assist you to test your code. You need to enable assertion in Java to enable the testing.
- **Ensure that you have passed all the test cases before submission, else zero marks will be awarded if the StudentAssertionTest.java failed to run.**
- Another comprehensive code will be used to automate the grading of your assignment.
- **Do not use any packages for the Java code as the automated grading will fail and will result in zero marks.**
- Include sufficient test cases to thoroughly test the classes and the methods implemented in your solution.

Submission:

- Please submit one zip file containing the following (do not place java class in a package):
 - AirQualityDataException.java
 - AirQualityReading.java
 - Analyser.java
 - CityAirStat.java
 - DistrictAirQualityAnalyser.java
- Only zip file is accepted. Please do not use other compression format such as 7z, gzip, rar and etc.
- For full time student, the zip file name must be in the form of: TXXF_NAME_UOWID.zip where XX is your tutorial group, NAME is your full name (without space and underscore) and UOWID is your 7-digit UOW ID (not SIM ID). For example: T01F_JohnTan_9876543.
- For part time student, the zip file name must be in the form of: TXXP_NAME_UOWID.zip where XX is your tutorial group, NAME is your full name (without space and underscore) and UOWID is your 7-digit UOW ID (not SIM ID). For example: T01P_JohnTan_9876543.
- Please ensure the UOWID is correct as it will be used by the marking scripts to update your assignment mark in Moodle. Incorrect ID will result in 0 mark in your Moodle.
- Wrong file naming (even an extra space, 5 marks will be deducted)
- Automated scripts will be used to extract your files for grading. Failure to follow the above instructions will result in failure to extract and grade your submission, and 0 mark will be awarded.
- No appeal will be granted for incorrect submission such as empty zip file, wrong files, etc.
- Late submission will be penalized 5% per day late.
- Late submission after 4 days will not be graded.

Appendices:

Sample data.csv file

```
2026-02-05,S001,North,35.2
2026-02-05,S002,North,40.6
2026-02-05,S003,North,38.1
2026-02-05,S001,South,55.0
2026-02-05,S002,South,48.3
2026-02-05,S003,South,52.4
2026-02-06,S001,North,30.4
2026-02-06,S002,North,28.8
2026-02-06,S003,North,33.5
2026-02-06,S001,South,45.1
2026-02-06,S002,South,47.6
2026-02-06,S003,South,44.0
#invalid date
2026-13-05,S101,South,42.0
#Invalid PM2.5
2026-02-06,S101,South,XYZ
#Invalid PM2.5 - out of range
2026-02-06,S101,South,-5.0
#multiple errors in one line: bad date + empty sensor id
2026-02-99,,South,520.0
```

Sample results.txt based on data.csv

```
2026-02-05_North : 37.97
2026-02-05_South : 51.9
2026-02-06_South : 45.57
2026-02-06_North : 30.9
```

Sample errors.txt based on data.csv

```
Line 14: Invalid Reading Date
Line 16: Invalid PM2.5
Line 18: Invalid PM2.5: -5.0
Line 20: Invalid Reading Date
Line 20: Invalid Sensor ID
```